Policies define the boundaries of behavior, promises communicate intent across different domains, and reputation tracks the results of those interactions. Together, these mechanisms allow independent systems—like ACME and its payment partner—to operate as a society of agents while still enforcing their own local policies.

Many contractor reimbursements were straightforward: a mileage claim, a reimbursable meal during travel, or a small software subscription approved for project work. Automating these transactions could greatly reduce manual effort. However, once authority moved from ACME's environment into the partner's payment system, ACME needed assurance that the partner's agent would operate within acceptable limits.

Trudy had already seen how quickly agent capabilities could expand inside ACME's systems. Past experiments demonstrated that agents might attempt unexpected actions unless their behavior was carefully constrained by policy. These lessons made her wary of granting authority outside the company's boundaries. If ACME was to let an external payment agent initiate reimbursements, she wanted assurance that the agent's actions would be governed by policies as strict as those used by ACME.

The partner's payment agent was also governed by Cedar policies. A simplified policy in the partner's domain might look like this:

```
permit (
    principal,
    action == Action::"reimbursement.pay",
    resource
)
when {
    context.delegation.capability == "contractor.reimbursement.pay"
    && context.payment.amount <= context.delegation.constraints.max_
amount
    && context.payment.currency == context.delegation.constraints.cu
rrency
    && context.payment.receipt_present == true
    && context.currentTime < context.delegation.constraints.expires
};
```